CFPB Complaint Classifier

Project Report and Technical Explanation

1. Introduction

1.1 What is the CFPB?

The Consumer Financial Protection Bureau (CFPB) is a United States government agency responsible for protecting consumers in the financial marketplace. It was established under the Dodd-Frank Wall Street Reform and Consumer Protection Act of 2010. The CFPB collects and analyzes consumer complaints about financial products and services including credit cards, mortgages, bank accounts, debt collection, and student loans. Each complaint includes a free-text narrative where the consumer describes their experience in their own words.

1.2 What Did We Build?

We built an end-to-end machine learning system that automatically classifies CFPB consumer complaints into two categories: Debt Collection and Credit Card. The system uses multiple approaches: four classical ML classifiers (Logistic Regression, Naive Bayes, Decision Tree, Random Forest) with TF-IDF text vectorization, cross-validation, and hyperparameter tuning, alongside Google Gemini 2.5 Flash as an LLM-based classifier. Results are served through a FastAPI REST backend and displayed in a Streamlit web interface.

1.3 Why Did We Build It?

Manually reading and routing thousands of free-text complaints daily is slow, expensive, and error-prone. An automated classifier can instantly triage complaints, reducing response time from hours to milliseconds while maintaining high accuracy. This type of system is directly applicable to any organization handling high volumes of unstructured text that needs to be categorized and routed to the correct team.

1.4 Interview Perspective

From an interview standpoint, this project demonstrates several key competencies: end-to-end ML pipeline design from raw data to production API, model selection and comparison using rigorous evaluation metrics (cross-validation, AUC-ROC, F1), handling class imbalance through stratified downsampling, building a REST API with FastAPI, creating a polished frontend with Streamlit, and integrating LLMs with traditional ML. It shows the ability to work across the full stack of an ML system rather than just training a model in a notebook.

2. Data Collection and Preprocessing

2.1 Raw Dataset

The raw dataset was obtained from the CFPB public complaint database. It contained approximately 115,000 complaints spanning multiple financial product categories. Each record includes a free-text consumer narrative, the product category, and metadata such as complaint date and company name.

2.2 Exploratory Data Analysis

During EDA we identified several issues: a significant class imbalance between Debt Collection and Credit Card complaints (approximately 2.2 to 1 ratio), 20,679 duplicate narratives that needed removal, and redacted text patterns (XXXX) representing personally identifiable information that had been scrubbed by the CFPB.

2.3 Preprocessing Steps

- Removed duplicate narratives and null entries
- Filtered to only Debt Collection and Credit Card categories
- Applied stratified downsampling to balance both classes at 4,000 samples each
- Split the balanced 8,000 samples 80/20 into training (6,400) and test (1,600) sets
- Applied TF-IDF vectorization with a maximum of 5,000 features

2.4 Why These Decisions Matter

Class imbalance causes models to be biased toward the majority class. Stratified downsampling ensures equal representation without generating synthetic data. Limiting TF-IDF to 5,000 features prevents overfitting on rare terms while retaining the most informative words. The 80/20 split provides enough test data for statistically meaningful evaluation while maximizing training data.

3. Model Training and Evaluation

3.1 Models Trained

Four classifiers were trained and evaluated:

- Logistic Regression - Linear model, fast training, strong baseline for text classification
- Naive Bayes - Probabilistic model, works well with TF-IDF features, very fast
- Decision Tree - Non-linear model, interpretable, prone to overfitting without tuning
- Random Forest - Ensemble of decision trees, reduces overfitting, robust performance

3.2 Training Process

Each model was trained using a Scikit-learn Pipeline combining TF-IDF vectorization and the classifier. Five-fold stratified cross-validation was used to get robust performance estimates. GridSearchCV was applied to each model to find optimal hyperparameters. The best parameters found were stored alongside the trained models.

3.3 Evaluation Metrics

- Accuracy - Overall correct prediction rate
- Precision - Of predicted positives, how many are actually positive
- Recall - Of actual positives, how many were correctly identified
- F1 Score - Harmonic mean of precision and recall, balances both
- AUC-ROC - Measures classification ability across all threshold settings

3.4 Results

Logistic Regression achieved the best performance with 93.75% accuracy, 93.75% F1 score, and 98.12% AUC. Random Forest followed with 91.95% F1 and 97.52% AUC. Naive Bayes reached 89.46% F1 and Decision Tree achieved 87.99% F1. The high AUC scores across all models indicate strong discriminative ability. The relatively small gap between models suggests the TF-IDF features are highly informative for this classification task.

4. System Architecture

4.1 FastAPI Backend

The backend is built with FastAPI, a modern Python web framework. It loads all four trained models and the TF-IDF vectorizer into memory on startup. The API exposes three endpoints: GET /health returns API status and model count, GET /models returns available models and their metrics from the training evaluation, and POST /predict accepts a complaint narrative and returns predictions from all four models with confidence scores. The API includes auto-generated OpenAPI documentation accessible at /docs.

4.2 Streamlit Frontend

The web interface is built with Streamlit using a custom dark theme. It connects to the FastAPI backend when available and falls back to loading models locally if the backend is offline. The interface shows predictions from all ML models side-by-side, Gemini LLM prediction with confidence and reasoning, and an Advanced Analytics section with confusion matrices, ROC curves, and feature importance plots.

4.3 LLM Integration

Google Gemini 2.5 Flash is used as a zero-shot classifier. The complaint narrative is sent to the model with a prompt requesting classification into Debt Collection or Credit Card along with confidence level and reasoning. This provides an independent comparison point against the trained ML models and adds explainability through the reasoning output.

4.4 Why This Architecture

Separating the backend API from the frontend allows independent scaling. The API can serve predictions to multiple clients (web app, mobile app, other services). FastAPI provides automatic input validation through Pydantic models and self-documenting API docs. The fallback to local models ensures the app works even without the backend running.

5. Conclusion and Interview Value

5.1 What This Project Demonstrates

- End-to-end ML pipeline from raw data to production deployment
- Model selection and comparison using rigorous statistical methods
- Handling real-world data issues (class imbalance, duplicates, missing values)
- Building production APIs with FastAPI including validation and documentation
- Creating polished user interfaces with Streamlit
- Integrating classical ML with modern LLMs
- Code organization and software engineering best practices

5.2 Key Technical Decisions

Using TF-IDF over word embeddings kept the model interpretable and fast to train. Choosing Logistic Regression as the best model was validated by both cross-validation and test set performance. Stratified downsampling was preferred over SMOTE because it does not generate synthetic text. The FastAPI backend was chosen over Flask for its automatic validation, async support, and built-in documentation.

5.3 Production Considerations

In a production environment, this system would need: a model registry for versioning, monitoring for data drift and model degradation, A/B testing between models, rate limiting on the API, authentication and authorization, and a CI/CD pipeline for automated retraining and deployment. The current architecture supports all of these additions without major refactoring.

5.4 What Would Improve It

- Model registry and versioning system
- Real-time monitoring dashboard for prediction distribution
- Automated retraining pipeline triggered by performance degradation
- Support for additional complaint categories beyond Debt Collection and Credit Card
- Unit and integration test suite
- Docker containerization for consistent deployment

5.5 Final Summary

This project shows the full lifecycle of an ML system: understanding the business problem, preprocessing messy real-world data, training and comparing multiple models with proper evaluation, building a REST API for serving predictions, and creating a polished frontend for end users. It demonstrates practical skills that are directly applicable to ML engineer and data scientist roles in industry.